

OPERATING SYSTEMS

Complete Study Notes

Process Management • Scheduling • Synchronization • Threads

Topics Covered

1. What is an Operating System?	6. Process States & Transitions
2. Goals & Parts of OS	7. Scheduling Queues & Schedulers
3. Dual Mode & System Calls	8. CPU Scheduling Algorithms
4. Process as a Data Structure	9. Threads & Multithreading
5. CPU Bound vs I/O Bound Processes	10. Synchronization & Critical Section
11. Peterson's Solution	12. Hardware Sync: TAS & Swap
13. Multilevel Queue Scheduling	

UNIT 1: What is an Operating System?

Definition

An Operating System (OS) is software that acts as an intermediary between the user/application programs and computer hardware.

More formally: An OS is a **resource manager** and **control program** that manages hardware resources and provides services to application programs.

Examples of Operating Systems:

Operating System	Description
Windows	Desktop/Laptop OS by Microsoft
Linux	Open-source OS used everywhere from phones to supercomputers
macOS	Apple's OS for Mac computers
Android	Linux-based mobile OS
iOS	Apple's mobile OS

Goals of Operating System

1. Convenience	Makes the system easy to use. Without OS, every program would need to communicate directly with hardware. With OS, a simple statement like <code>cout << "Hello";</code> handles complex display hardware internally.
2. Efficiency	Utilizes CPU, memory, disk, and I/O devices efficiently. CPU Scheduling algorithms improve CPU utilization — preventing the processor from sitting idle.
3. Portability	Applications should run on different hardware with minimal changes. Example: Google Chrome runs on Windows, Linux, and macOS without rewriting.
4. Reliability	System should work correctly without crashing frequently. Memory protection prevents one process from corrupting another process's memory space.
5. Scalability	Should work efficiently as resources increase. Linux, for instance, can run on mobile phones, personal computers, and supercomputers alike.

6. Robustness

Ability to handle failures gracefully. If one application crashes, the whole OS should not crash — other processes keep running.

Parts of Operating System

A. The Kernel

The Kernel is the **core part** of the Operating System. It contains almost all major functionalities of the OS and directly communicates with hardware — applications cannot.

Kernel Responsibilities:

Kernel Responsibility	Description
Process Management	Creating, scheduling, and terminating processes
Memory Management	Allocating and freeing memory
CPU Scheduling	Deciding which process runs next
Device Management	Managing I/O devices via drivers
File System Management	Create, read, write, delete files on disk
Security & Protection	Preventing unauthorized resource access
Interrupt Handling	Responding to hardware/software interrupts

Architecture: User → Application → Kernel → Hardware

The kernel sits between applications and hardware. Applications call the kernel via System Calls; the kernel then interacts with hardware directly.

B. The Shell

Shell acts as an interface between the User and the Kernel. It interprets user commands and passes them to the kernel for execution.

Shell Type	Examples & Description
CLI (Command Line Interface)	Bash, Zsh, PowerShell — user types commands like <code>ls</code> , <code>pwd</code> , <code>mkdir</code>
GUI (Graphical User Interface)	Windows Desktop, Android UI, macOS Finder — user clicks buttons and icons

UNIT 2: Dual Mode Operation & System Calls

Dual Mode Operation

Modern Operating Systems run in two modes to ensure protection and security.

Property	User Mode	Kernel Mode
Mode Bit Value	1	0
Mode Name	User Mode	Kernel Mode
Who Executes	Application programs (Chrome, VS Code, WhatsApp)	Operating System itself
Hardware Access	■ Cannot access hardware directly	■ Full access to CPU, Memory, Disk, I/O
Memory Access	■ Cannot modify kernel memory	■ Complete access
Privileged Instructions	■ Cannot execute	■ Can execute

Why Two Modes? — Protection Example

Imagine a bug in Chrome that runs: `while(true){ write_kernel_memory(); }`

Without protection, this would crash the **entire OS**. With dual-mode, user programs run with limited privileges — a bug can only damage its own process, not the kernel.

System Calls

Definition: A System Call is a mechanism through which a user program requests a service from the Operating System Kernel.

System Calls are the **interface between User Space and Kernel Space**. User programs cannot access hardware directly — they must request the kernel through system calls.

System Call Flow:

User Process → System Call → Kernel → Device Driver → Hardware

File Open: `FILE* fp = fopen("abc.txt", "r");`

Application → open() → System Call → Kernel → File System → Disk

Keyboard Input: `cin >> x;`

Program → read() → System Call → Kernel → Keyboard Driver → Keyboard Hardware

Process Creation: `fork();`

Process creation requires kernel involvement — kernel manages process tables

Network Communication: `send(); recv();`

These are system calls — Kernel interacts with network hardware

Display Output: `printf("Hello");`

Application → write() → System Call → Kernel → Display Driver → Screen

Program Execution — Most Important OS Service

Any Operating System MUST support Program Execution — without this, an OS has no practical use.

When you double-click Chrome.exe, the OS performs 6 key steps:

Step 1	Load Executable from Disk — Find and read the program file from storage
Step 2	Create Process Control Block (PCB) — OS creates a data structure to track this process
Step 3	Allocate Memory — Reserve RAM for code, data, heap, and stack
Step 4	Initialize CPU Registers — Set Program Counter to the first instruction
Step 5	Schedule Process on CPU — Add to ready queue; scheduler will pick it
Step 6	Start Execution — CPU begins executing program instructions

Services Provided by the Operating System

OS Service	Description
User Interface	CLI / GUI — ways for users to interact with the system
Program Execution	Load and run application programs
I/O Operations	Keyboard, Mouse, Printer, Monitor management
File System Manipulation	Create, delete, read, write, rename files
Inter-Process Communication (IPC)	Pipes, Shared Memory, Message Queues, Sockets
Error Detection	Detect memory errors, hardware failures, disk errors
Resource Allocation	Allocate CPU, Memory, Disk, Network fairly
Accounting	Track resource usage (visible in Task Manager)
Protection & Security	Passwords, permissions, memory protection

UNIT 3: Types of Operating Systems

1. Uni-Programming OS

Definition: Only **one process** resides in main memory and executes at a time.

- Memory layout: OS + Single Process P1
- Problem: When P1 performs I/O (e.g., read from file), the CPU remains IDLE
- Result: Poor CPU utilization — CPU Busy means I/O Idle, and vice versa
- Example: Early systems like MS-DOS

2. Multi-Programming OS

Definition: Multiple processes reside in main memory simultaneously. CPU switches to another process whenever one is waiting.

- Memory: OS + P1 + P2 + P3 + P4 all loaded together
- When P1 waits for I/O, CPU executes P2 — CPU never sits idle
- Degree of Multiprogramming = number of processes in RAM
- Higher degree generally means better CPU utilization

Non-Preemptive: CPU cannot be forcibly taken. Process runs until it terminates or requests I/O. Examples: FCFS, Non-preemptive SJF

Preemptive: OS can forcibly remove a process from CPU (e.g., time quantum expires). Examples: Round Robin, Preemptive Priority

3. Multi-Tasking OS

Definition: Extension of Multiprogramming where processes share CPU through **time slicing**.

- Uses Round Robin scheduling with a fixed time quantum
- CPU timeline: P1|P2|P3|P4|P1|P2... (e.g., 5ms each)
- Switching is so fast users feel all processes run simultaneously
- Example: Running Chrome + VS Code + Spotify at the same time

4. Multi-User OS

Definition: Allows multiple users to access the same computer system simultaneously.

- Resources shared: CPU, Main Memory, Disk, Network
- OS responsibilities: user authentication, access control, fair allocation
- Example: Linux Server — many users connect via SSH simultaneously
- University lab server where hundreds of students login at once

5. Multi-Processing OS

Definition: System contains multiple CPUs/Processors working together.

- Higher throughput, faster execution, better reliability

- Tightly Coupled (Shared Memory): CPU1, CPU2, CPU3 share same RAM — fast communication.
Example: Modern multicore Intel i7, AMD Ryzen
- Loosely Coupled (Distributed): Each CPU has its own memory, communicate via network. Example:
Google Search infrastructure

6. Real-Time OS (RTOS)

Definition: Used when tasks MUST complete within strict deadlines. Correctness depends on both the result AND the timing.

- Every task has a deadline — missing it may cause system failure
- Applications: Air Traffic Control, Medical Equipment, Missile Systems, Robotics, Automotive, Industrial Automation
- Hard RTOS: Missing deadline = System Failure (Aircraft controls, Nuclear reactor)
- Soft RTOS: Missing deadline = Performance degrades but system works (Video streaming, Online gaming)

7. Embedded OS

Definition: OS designed for a specific purpose/device — NOT a general-purpose OS.

- Characteristics: Small memory footprint, lightweight, highly reliable, dedicated functionality
- Examples: Washing machines (water+motor+timer control), Microwave, Printer
- Embedded OS Examples: FreeRTOS, Embedded Linux, VxWorks, QNX

8. Hand-Held Device OS

Definition: Operating systems designed for portable handheld devices.

- Characteristics: Battery efficient, touch support, wireless connectivity, resource optimized
- Android: Uses Linux kernel
- iOS: Uses XNU kernel
- Challenges: Limited battery, limited memory, mobile networking, sensor management

UNIT 4: Process as a Data Structure

Definition of a Process

Simple Definition: A Process is a **Program Under Execution**.

A program sitting on disk is just passive code. When you double-click it and the CPU starts executing: memory gets allocated, registers are assigned — **now it becomes a process**.

Process = Program + Runtime Activity

Other Definitions of Process:

Definition	Explanation
Program Under Execution	Most common definition — what runs on the CPU
Instance of a Program	Chrome.exe opened 5 times = 1 program but 5 processes
Schedulable / Dispatchable Unit	The OS scheduler schedules processes, not programs
Unit of Execution	CPU executes processes — the atomic unit of work
Locus of Control	Contains current instruction, state, and resources

Process Memory Layout

A process in memory is divided into four sections:

Stack	Local variables, function parameters, return addresses. Grows/shrinks dynamically with function calls. Auto-freed on return.
Heap	Dynamic memory allocation (new, malloc). Size decided at runtime. Grows upward. Programmer manages this.
Data Section	Global variables and static variables. Lifetime = entire process lifetime. Created when process starts, destroyed when it ends.
Code / Text Section	Program instructions (executable code). Read-only. Fixed size after compilation.

Important Interview Point: Why Global and Static Variables both in Data Section?

Both global and static variables share the same property: their **lifetime = entire process lifetime**.

- Global variable: accessible throughout the program. Lifetime = entire process.
- Static variable: scope may be local, but lifetime is entire process — even after the function returns, the variable survives.

Key Rule: Storage location depends on LIFETIME, not on ACCESSIBILITY.

Stack vs Heap Comparison

Property	Stack	Heap
Allocation	Automatic	Manual (new/malloc)
Stores	Local variables, function params	Dynamic objects, arrays
Speed	Very Fast	Slower
Memory Freed	Automatically on function return	Programmer must free
Size Limit	Fixed limits	Can grow dynamically
Example	<code>int x = 10;</code> inside a function	<code>int *arr = new int[n];</code>

Attributes of a Process

Attribute	Description
PID (Process ID)	Unique number assigned by OS to identify each process. No two running processes share the same PID.
PC (Program Counter)	Stores the ADDRESS of the next instruction to execute. Without PC, the CPU won't know where to continue.
GPR (General Purpose Registers)	CPU registers (AX, BX, CX, RAX, etc.) storing data, operands, intermediate results.
List of Devices	I/O devices allocated to the process (Printer, Scanner, Keyboard, GPU).
Type	System Process, User Process, Interactive Process, Batch Process.
Size	Memory (in bytes) currently occupied by the process.
Memory Limits	Maximum memory the process is allowed to use.
Priority	Used by scheduler — higher priority processes may get CPU first.
State	Current state: New, Ready, Running, Waiting, or Terminated.
List of Open Files	Files currently opened by the process (report.pdf, data.txt, etc.)

Process Control Block (PCB)

PCB is a data structure maintained by the OS for every process — think of it as the Process's Identity Card.

Contents: PID, Program Counter, CPU Registers, Priority, State, Open Files, Memory Information.

Key Formula: Context of a Process = Contents of its PCB = Entire Execution State.

Context Switching

When the CPU must switch from Process P1 to Process P2:

Step	Action
Step 1	Save P1 Context — OS stores P1's PC, SP, and Registers into P1's PCB
Step 2	Load P2 Context — OS loads P2's PC, SP, and Registers from P2's PCB into CPU
Step 3	P2 resumes execution from exactly where it left off

Scheduler = Decides WHICH process should run next.

Dispatcher = Actually performs the context switch (saves/loads state, transfers CPU control).

Operations on a Process

Operation	Description
Create	OS allocates memory, CPU structures, and PCB. Resources assigned.
Schedule / Run	Scheduler selects; Dispatcher transfers CPU. Process starts running.
Wait / Block	Process requests I/O or event. State: Running → Waiting.
Suspend / Resume	OS temporarily stops process (swaps out) and later resumes it.
Terminate	Process completes. Resources released, memory freed, files closed, PCB deleted.

UNIT 5: CPU Bound vs I/O Bound Processes & Process States

CPU Bound vs I/O Bound Processes

Property	CPU Bound Process	I/O Bound Process
Definition	Spends most time doing CPU computations	Spends most time waiting for I/O devices
CPU Bursts	Long CPU bursts	Short CPU bursts
I/O Requests	Few I/O requests	Frequent I/O requests
CPU Utilization	Very High	Low
Waiting State	Rarely enters waiting state	Frequently enters waiting state
State Behavior	Ready → Running → Ready (cycle)	Running → Waiting → Ready → Running
Real Examples	Matrix multiplication, AI/ML training, Video rendering	File reads, Browser downloading, Database queries

Process States

A process moves through different states during its lifetime. Understanding these transitions is critical for OS interviews.

NEW	Process has just been created. OS is creating PCB and allocating initial resources. The program is transitioning into a process.
READY	Process is ready to execute. Memory allocated, resources allocated, waiting ONLY for CPU. Process sits in the Ready Queue.
RUNNING	CPU is currently executing process instructions. For a single-core CPU, only ONE process can be in this state at a time.
WAITING / BLOCKED	Process cannot continue until an event occurs (disk read complete, keyboard input received, network response, printer done). Caused by the process itself requesting I/O.
TERMINATED	Execution completed. Resources released: memory freed, files closed, PCB deleted. Triggered by return 0; or exit(0);

SUSPENDED READY	Process is ready (needs only CPU) BUT has been swapped out of main memory to disk by the Mid-Term Scheduler.
SUSPENDED BLOCKED	Process is blocked (waiting for I/O) AND has been swapped out of main memory to disk.

Process State Transitions

Transition	Caused By	Reason
New → Ready	OS (Long-Term Scheduler)	Process admitted; PCB created, memory allocated, added to Ready Queue
Ready → Running	OS (Scheduler + Dispatcher)	Scheduler selects; Dispatcher performs context switch and gives CPU
Running → Terminated	Process Itself	Execution finished (return 0; or exit(0))
Running → Waiting	Process Itself	Process voluntarily requests I/O or waits for event
Running → Ready	OS (Preemption)	Time quantum expired OR higher-priority process arrived — CPU forcibly taken
Waiting → Ready	OS (Interrupt Handler)	Requested event completed (I/O done, input received)
Ready → Suspended Ready	Mid-Term Scheduler	Swapped out of RAM to disk to free memory
Suspended Ready → Ready	Mid-Term Scheduler	Swapped back into RAM
Waiting → Suspended Blocked	Mid-Term Scheduler	Swapped out while still waiting for I/O
Suspended Blocked → Waiting	Mid-Term Scheduler	Swapped back in; still waiting for I/O

Preemption = OS forcibly taking CPU from a running process (e.g., time quantum expired).

Key Rule: A process can NEVER go directly from Waiting → Running. It must go Waiting → Ready → Running.

Only Running state actually uses the CPU. Ready = waiting for CPU. Waiting = waiting for I/O/event.

UNIT 6: Scheduling Queues, Schedulers & Swapping

Scheduling Queues

Queue	Process State	Managed By	Description
Job Queue	New State	Long-Term Scheduler	All processes created but not yet in RAM. Stored on disk.
Ready Queue	Ready State	Short-Term Scheduler	Processes in RAM, waiting only for CPU.
Device Queue	Waiting State	N/A (device drivers)	Processes waiting for a specific I/O device. Each device has its own queue.

Flow: Job Queue → Ready Queue → Running → Device Queue → Ready Queue

Three Types of Schedulers

Long-Term Scheduler (also: Job Scheduler)

Moves: Job Queue → Ready Queue (New → Ready)

Frequency: Low — runs rarely (process admission is infrequent)

Main Goal: Controls MAXIMUM degree of multiprogramming — decides how many processes enter RAM

Short-Term Scheduler (also: CPU Scheduler)

Moves: Ready Queue → Running (Ready → Running)

Frequency: Very High — possibly every few milliseconds

Main Goal: Selects which ready process gets CPU next. CPU cannot remain idle.

Mid-Term Scheduler (also: Medium-Term Scheduler)

Moves: RAM ↔ Secondary Storage (Swapping)

Frequency: Medium

Main Goal: Performs swapping. REDUCES degree of multiprogramming by removing processes from RAM.

Scheduler vs Dispatcher — Common Confusion

Property	Scheduler	Dispatcher
Role	Selects which process runs next	Actually gives CPU to selected process
Action	Decision-making only	Context switch + mode switch + CPU transfer

Analogy	Teacher selects the student to answer	Teacher hands the microphone to that student
Output	A process chosen from Ready Queue	CPU now executing the chosen process

Swapping (Mid-Term Scheduler)

Swapping = Temporarily moving a process from Main Memory to Secondary Storage and bringing it back later. Also called **Rolling**.

Swap-Out: RAM → Disk (process moved to free memory for others)

Swap-In: Disk → RAM (process brought back when needed)

Why Swapping? RAM shortage, reduce degree of multiprogramming, allow more processes overall.

Interview Question	Answer
Which scheduler controls MAX degree of multiprogramming?	Long-Term Scheduler
Which scheduler REDUCES degree of multiprogramming?	Mid-Term Scheduler
Which scheduler selects process for CPU?	Short-Term Scheduler
Who performs context switching?	Dispatcher
What is Suspended Ready?	Ready but NOT in RAM (swapped to disk)
What is Suspended Blocked?	Blocked AND not in RAM (swapped while waiting for I/O)

UNIT 7: CPU Scheduling Algorithms

Key Scheduling Terminologies

Term	Definition / Formula	Notes
Arrival Time (AT)	Time at which process enters system and is ready	Process arrives at time 5 $\rightarrow$ AT = 5
Burst Time (BT)	Total CPU time needed by process to complete	Needs 8 units of CPU $\rightarrow$ BT = 8
Completion Time (CT)	Time at which process finishes execution	Also called Exit Time or Finish Time
Turnaround Time (TAT)	Total time from arrival to completion: $TAT = CT - AT = WT + BT$	Includes waiting + execution time
Waiting Time (WT)	Time spent in Ready Queue: $WT = TAT - BT = CT - AT - BT$	Does NOT include execution time
Response Time (RT)	Time from arrival until FIRST CPU allocation: $RT = \text{First CPU Time} - AT$	Critical for interactive systems
Throughput	Processes completed per unit time = $\frac{\text{Processes}}{\text{Total Time}}$	Higher throughput = better performance
Scheduling Length (L)	$L = \max(CT) - \min(AT)$	Total span of scheduling period

1. FCFS — First Come First Serve

Type: Non-Preemptive | **Selection Criterion:** Arrival Time (AT) — smallest AT executes first

Tie Breaker: Smaller Process ID

Core Idea: The process that arrives first gets CPU first — exactly like a queue.

Advantages:

- Very simple to implement using a queue
- Fair — processes served in order of arrival
- No starvation — every process eventually gets CPU

Disadvantages:

- Convoy Effect: a long process delays all shorter processes behind it
- Poor Average Waiting Time for short jobs
- Poor Response Time — not suitable for interactive systems

2. SJF — Shortest Job First

Type: Non-Preemptive | **Selection Criterion:** Burst Time (BT) — smallest BT executes first

Tie Breaker: FCFS (arrival time)

Core Idea: The process with the smallest Burst Time is executed first. Gives MINIMUM average waiting time among all non-preemptive algorithms.

Advantages:

- Optimal: gives minimum average waiting time
- Better turnaround time compared to FCFS
- Short jobs finish quickly

Disadvantages:

- Starvation: long jobs may wait indefinitely if short jobs keep arriving
- Burst Time prediction required — actual BT usually unknown beforehand
- Not suitable for interactive systems

3. SRTF — Shortest Remaining Time First

Type: Preemptive (Preemptive SJF) | **Selection Criterion:** Remaining Burst Time — process with smallest remaining time runs

Tie Breaker: FCFS

Core Idea: At every instant, execute the process with smallest remaining burst time. Whenever a new process arrives, compare its BT with running process's remaining time — preempt if new process is shorter.

Advantages:

- Better average WT than non-preemptive SJF
- Better response time
- Favors short jobs

Disadvantages:

- Starvation still possible for long processes
- High context switching overhead
- More complex to implement

4. HRRN — Highest Response Ratio Next

Type: Non-Preemptive | **Selection Criterion:** Response Ratio = $(WT + BT) / BT = 1 + (WT/BT)$ — Highest RR executes first

Tie Breaker: Smaller BT preferred

Core Idea: Designed to solve the starvation problem of SJF. As a process waits longer, its Response Ratio increases — eventually it gets CPU. Balances short jobs and long waiting jobs.

Advantages:

- Prevents starvation — waiting time increases RR automatically
- Good average waiting time
- Balances short and long jobs fairly

Disadvantages:

- More calculation required each time (compute RR for all)
- Less practical — mostly used in theory

5. Priority Scheduling

Type: Non-Preemptive OR Preemptive | **Selection Criterion:** Priority — higher priority process executes first

Tie Breaker: FCFS

Core Idea: CPU allocated according to process priority. Preemptive variant: newly arrived higher-priority process can preempt the currently running one.

Advantages:

- Important processes execute earlier
- Flexible scheduling policy
- Useful in Real-Time Systems

Disadvantages:

- Starvation: low-priority processes may wait forever
- Priority Inversion issues possible
- Solution to starvation: Aging — gradually increase priority of waiting processes

6. Round Robin (RR)

Type: Preemptive | **Selection Criterion:** Arrival Time + Circular Queue Rotation with Time Quantum Q

Tie Breaker: Process ID

Core Idea: Each process gets CPU for a fixed Time Quantum Q. After Q expires, process is preempted and sent to end of Ready Queue. Achieves fairness and interactiveness.

Advantages:

- Fair — every process gets CPU periodically
- Good response time for interactive systems
- No starvation

Disadvantages:

- Very small Q → excessive context switching, CPU efficiency ≈ 0
- Very large Q → RR becomes equivalent to FCFS
- Performance heavily depends on choosing the right Q

Round Robin — Effect of Time Quantum Size

Quantum Size	Effect
Very Small Q	Excessive context switching → CPU efficiency ≈ 0 → Throughput drops
Small Q	Highly interactive, good response time, but more context switches
Large Q	Less interactive, response time increases
Very Large Q	RR $\equiv$ FCFS — processes finish before quantum expires

UNIT 8: Threads & Multithreading

Why Do We Need Threads?

When multiple execution flows are needed, creating multiple full processes wastes memory (each has its own code, data, files, OS resources).

Solution: Create lightweight execution units called THREADS that share process resources but maintain separate execution contexts.

Thread = Lightweight Process = Smallest unit of CPU execution = Component of a process

Process vs Thread — Anatomy

Property	Process	Thread
Type	Heavyweight entity	Lightweight entity
Contains	Address Space, Resources, Files, Security Info, PCB	Thread ID, Program Counter, Registers, Stack
Memory	Has own address space	Shares process address space
Creation Cost	Expensive (full resource allocation)	Cheap (only execution context)
Communication	Requires IPC (pipes, sockets)	Shared memory already exists
Context Switch	Heavy (address space, page tables, resources)	Light (PC, registers, stack only)

What Each Thread Shares vs Owns

Resource	Status	Why
Code Section	■ Shared	Only one copy — all threads execute same instructions
Data Section	■ Shared	Global variables visible to all threads
Heap	■ Shared	Dynamic memory shared by all threads
Open Files	■ Shared	File descriptors accessible by all threads
OS Resources	■ Shared	Sockets, memory mappings, signals
Thread ID	■ Unique per thread	Identifier for this specific thread
Program Counter	■ Unique per thread	Each thread knows its own next instruction
Register Set	■ Unique per thread	CPU registers (AX, BX, etc.) separate per thread
Stack	■ Unique per thread	Each thread needs own local variables — critical!

Advantages of Multithreading

Advantage	Explanation
Responsiveness	UI remains responsive while background work continues (Word saves file while accepting keyboard input)
Faster Context Switch	Thread switch involves only PC, registers, stack — much lighter than process switch (address space unchanged)
Resource Sharing	Threads share memory/files — no IPC needed, communication is just shared variables
Economy	Creating threads is cheaper: less memory, less OS overhead
Easy Communication	Shared memory already exists — no pipes or sockets needed
Better Multiprocessor Utilization	4 threads on 4 cores run truly in parallel — maximum CPU usage

User Level Thread (ULT) vs Kernel Level Thread (KLT)

Property	User Level Thread (ULT)	Kernel Level Thread (KLT)
Managed By	Thread library (POSIX pthreads, Green Threads)	Operating System / Kernel
Kernel Awareness	Kernel is UNAWARE — sees only one process	Kernel is FULLY AWARE — tracks each thread
Speed	Very Fast — no system call, no mode switch	Slower — requires system calls
Blocking Behavior	One thread blocks → ENTIRE PROCESS blocks (critical flaw!)	One thread blocks → only that thread blocks
Parallelism	No true parallelism — one kernel thread	True parallelism on multicore CPUs
Portability	Portable across OS	OS-dependent
Cost	Cheap	More memory/overhead
Data Structure	Thread info in user-space library	Thread Control Block (TCB) in kernel

Critical Flaw of ULT:

Since the kernel is unaware of individual threads, if Thread T1 makes a blocking system call `read()`, the kernel blocks the ENTIRE PROCESS.

Result: T2 and T3 also get blocked, even though they have nothing to do with the I/O.

This is why KLT is preferred in modern systems despite being heavier.

Multithreading Models (ULT to KLT Mapping)

Many-to-One Model

Many user threads mapped to ONE kernel thread.

- Very fast — user-space scheduling
- No true parallelism — only one kernel thread
- One thread blocks → entire process blocks
- Cannot utilize multicore CPUs

One-to-One Model

Each user thread has its own dedicated kernel thread. Used in: Windows, Linux, Modern Systems.

- True parallelism on multicore CPUs
- Better responsiveness
- Individual blocking — one thread blocked doesn't affect others
- Drawback: More memory and kernel overhead per thread

Many-to-Many Model

Many user threads mapped onto fewer or equal kernel threads. Combines advantages of both models.

- User-level flexibility
- Kernel-level parallelism
- No strict one-to-one overhead
- Most flexible model

Thread Context Switching:

When CPU switches Thread A → Thread B:

Step 1: Save Thread A's PC, Registers, Stack Pointer into Thread A's TCB.

Step 2: Load Thread B's PC, Registers, Stack Pointer from Thread B's TCB.

Step 3: CPU resumes Thread B from where it left off.

Much cheaper than process context switching because the address space remains the same — no page table changes.

UNIT 9: Process Synchronization & Race Condition

Why Synchronization is Needed?

Synchronization is necessary when multiple processes or threads access the same **shared resource** (variable, memory, file, database record, printer, buffer).

Independent Process: Does not share data — no synchronization needed. (e.g., Calculator + Music Player)

Cooperating Process: Shares information — synchronization **MANDATORY**. (e.g., Producer + Consumer sharing a buffer)

Race Condition

Definition: A race condition is an undesirable situation in which the final outcome depends on the ORDER or TIMING of execution of concurrent processes/threads.

Two or more processes 'race' to modify the same data — whoever finishes last determines the result.

A race condition requires **ALL FOUR** of these conditions:

Condition	Example
1. Shared Resource Exists	e.g., <code>int count</code> ; accessible by multiple threads
2. Multiple Threads Access It	T1, T2, T3 all reading/writing count
3. At Least One Thread Modifies	<code>count++</code> ; is a write operation
4. Final Result Depends on Timing	Different execution orders → different outputs

Concrete Race Condition Example:

Shared variable: `int a = 5`; — P1 adds 2, P2 adds 5. Expected final result: 12

Dangerous scenario:

P1 executes: `R1 = a` → `R1 = 5`. Then CPU preempts P1.

P2 executes: `R3 = a` → `R3 = 5`. Then `R3 = 10`, `a = 10`.

P1 resumes: `R1` was still 5! So `R1 = 7`, `a = 7`. P2's update is LOST!

Final result: `a = 7` instead of expected 12 — data corruption!

Problems caused by Race Conditions:

- Inconsistency — different outputs every run
- Data Loss — some writes disappear (`count++` executed twice but result is 1, not 2)
- System Crash — corrupted data structures (linked list corruption)
- Deadlock — improper synchronization may lead to processes waiting forever

- Starvation — one process never gets CPU/resource

Critical Section

Definition: A Critical Section (CS) is a part of a program where shared resources are accessed or modified.

Important: Race conditions occur ONLY inside critical sections.

Example: `balance = balance - 100;` — accesses shared variable → critical section.

Critical Section Structure:

Section	Purpose & Example
Entry Section	Request permission to enter CS. e.g., <code>lock.acquire();</code>
Critical Section	Access/modify shared resource. e.g., <code>balance -= 100;</code>
Exit Section	Release permission. e.g., <code>lock.release();</code>
Remainder Section	Remaining code that does NOT access shared resources.

Why does `count++` need protection?

Although it looks like one statement, the CPU executes THREE separate instructions:

1. LOAD count into register
2. INCREMENT register value
3. STORE register back to count

A context switch between any of these steps causes data loss.

Requirements of Any Synchronization Solution

Requirement	Meaning	Violation Example
1. Mutual Exclusion	At any moment, ONLY ONE process can execute inside the CS. If P0 is in CS, P1 must wait. Most important requirement.	■ Two processes in CS simultaneously
2. Progress	If NO process is in CS and some want to enter, one MUST be allowed immediately. Processes not interested in CS must NOT affect the decision.	■ CS empty but nobody allowed in
3. Bounded Waiting	A process must NOT wait forever. There must be a finite limit on how many times others can enter before it gets its turn. Prevents starvation.	■ P0 waits forever while P1 enters repeatedly

UNIT 10: Critical Section Solutions & Peterson's Algorithm

Solution 1: Lock Variable

Variables: `Boolean lock = false;` (false = CS free, true = CS occupied)

Core Idea: Use a shared boolean. Process checks lock before entering, sets it to true, then resets to false on exit.

Code:

```
while(lock);  
lock = true;  
// Critical Section  
lock = false;
```

Why It Works / Why It Fails: The lock itself is a shared variable. Checking and setting are TWO SEPARATE operations — not atomic. A process can be preempted AFTER checking (lock=false) but BEFORE setting (lock=true), allowing another process to also see lock=false and enter. Result: TWO processes inside CS simultaneously.

Mutual Exclusion: ■ Fails | **Progress:** ■ Passes | **Bounded Waiting:** ■ Not Guaranteed

Solution 2: Turn Variable (Strict Alternation)

Variables: `int turn = 0;` (0 = P0's turn, 1 = P1's turn)

Core Idea: Use a shared turn variable. Processes execute in strict alternating order: P0 → P1 → P0 → P1...

Code:

```
// P0  
while(turn != 0);  
// CS  
turn = 1;  
  
// P1  
while(turn != 1);  
// CS  
turn = 0;
```

Why It Works / Why It Fails: Progress is violated. If turn=0 but P0 is in Remainder Section (not interested in CS), P1 CANNOT enter even though CS is empty. A process not interested in CS is blocking another. Processes must alternate strictly even if one has no work.

Mutual Exclusion: ■ Passes | **Progress:** ■ Fails | **Bounded Waiting:** ■ Passes

Solution 3: Peterson's Algorithm ■

Variables: `boolean flag[2] = {false, false}; int turn;`

Core Idea: Combines flag (intention) and turn (priority). flag[i]=true means Pi WANTS to enter CS. turn gives priority when both want CS simultaneously.

Code:

```
// Process P0
flag[0] = true;
turn = 1;
while(flag[1] && turn==1);
// Critical Section
flag[0] = false;

// Process P1
flag[1] = true;
turn = 0;
while(flag[0] && turn==0);
// Critical Section
flag[1] = false;
```

Why It Works / Why It Fails: Peterson's satisfies ALL THREE requirements — it does not fail.

Mutual Exclusion: ■ Passes | **Progress:** ■ Passes | **Bounded Waiting:** ■ Passes

Peterson's Solution — How It Handles All Cases

Scenario	Explanation
Only P0 wants CS	flag[0]=true, flag[1]=false → while(flag[1] && turn==1) is false immediately → P0 enters CS
Only P1 wants CS	flag[1]=true, flag[0]=false → P1 enters immediately
Both want CS simultaneously	Both flags true. Last writer of 'turn' gives priority to OTHER process. One waits, one enters. After completion, flag[i]=false and waiter proceeds.
How ME is ensured	If both in entry: one will find the while-condition true and spin. They can't both exit the while loop.
How Progress is ensured	Processes not interested don't set their flag → others can enter freely.
How BW is ensured	After P0 finishes, P0 sets flag[0]=false. P1 (if waiting) can now enter. P0 can only re-enter by setting turn=1 first, giving P1 priority.

Final Comparison: All Three Solutions

Solution	Key Mechanism	Mutual Exclusion	Progress	Bounded Waiting
Lock Solution	Shared lock — not atomic	■	■	■
Turn Variable	Strict alternation	■	■	■
Peterson's Solution	flag[] + turn — software solution for 2 processes	■	■	■

Memory Trick: Lock → Shared lock → Mutual Exclusion FAILS | Turn → Strict alternation → Progress FAILS | Peterson → Flag + Turn → ALL THREE SATISFIED

Limitation: Peterson's Solution works for only TWO processes.

UNIT 11: Synchronization Hardware — TestAndSet() & Swap()

Why Hardware Solutions?

The Lock Variable solution failed because checking and setting the lock were two SEPARATE non-atomic operations.

Solution: Hardware-supported CPU instructions that execute as a single INDIVISIBLE (atomic) unit.

Atomic Operation: Executes completely or not at all. No context switch can occur in the middle.

1. TestAndSet() Instruction

Returns the current value of a variable AND simultaneously sets it to TRUE — atomically.

Pseudocode:

```
Boolean TestAndSet(Boolean *target)
{
    Boolean rv = *target;
    *target = true;
    return rv;
}
```

Usage in Critical Section:

```
Boolean Lock = false; // shared

while(true) {
    while(TestAndSet(&Lock;)); // spin until lock acquired
    // Critical Section
    Lock = false; // release lock
}
```

How TestAndSet() Works:

Scenario	Result
Lock = false (CS free)	TAS returns false, sets Lock=true. while(false) → exit loop → ENTER CS
Lock = true (CS occupied)	TAS returns true, keeps Lock=true. while(true) → keep spinning (Busy Wait)
After CS done	Set Lock=false → waiting process acquires on next TAS call

Why TAS works: Checking and setting happen ATOMICALLY — no preemption possible between them.

Busy Waiting / Spin Lock: Waiting processes keep executing `while(TestAndSet(&Lock;));` — wasting CPU cycles.

Properties: ■ Mutual Exclusion ■ Progress ■ Bounded Waiting (starvation possible)

2. Swap() Instruction

Exchanges the values of two variables ATOMICALLY.

Pseudocode:

```
void Swap(Boolean *a, Boolean *b)
{
    Boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

Usage in Critical Section:

```
Boolean Lock = false; // shared
Boolean Key; // local to each process

while(true) {
    Key = true;
    while(Key == true)
        Swap(&Lock;, &Key;);
    // Critical Section
    Lock = false;
}
```

How Swap() Works:

Scenario	Result
Lock=false, Key=true	After Swap: Lock=true, Key=false → while(Key==true) is false → ENTER CS
Lock=true, Key=true	After Swap: Lock=true, Key=true → while(Key==true) stays true → keep spinning
Why it works	Swap is atomic — no other process can interleave between checking and acquiring the lock

TestAndSet vs Swap — Comparison

Property	TestAndSet()	Swap()
----------	--------------	--------

Hardware Support	■ Yes	■ Yes
Atomic	■ Yes	■ Yes
Mutual Exclusion	■ Satisfied	■ Satisfied
Progress	■ Satisfied	■ Satisfied
Bounded Waiting	■ Not Guaranteed	■ Not Guaranteed
Busy Waiting	■ Present (spin lock)	■ Present (spin lock)
Starvation Possible	■ Yes	■ Yes
Variables Used	Boolean Lock	Boolean Lock + Boolean Key

Complete Comparison: All Synchronization Methods

Method	ME	Progress	BW	Key Note
Lock Variable	■	■	■	Two separate ops — not atomic
Turn Variable	■	■	■	Strict alternation blocks progress
Peterson's Solution	■	■	■	Software — 2 processes only
TestAndSet()	■	■	■	Hardware — busy waiting
Swap()	■	■	■	Hardware — busy waiting

UNIT 12: Multilevel Queue (MLQ) Scheduling

Why MLQ is Needed?

Different types of processes have fundamentally different requirements:

- System Processes → require very fast response and highest priority
- Interactive/Foreground Processes → need quick user response
- Background/Batch Processes → focus on throughput, less sensitive to response time

Problem: A single scheduling algorithm cannot efficiently satisfy all these needs simultaneously.

Solution: Divide the ready queue into multiple separate queues, each for a specific category with its own algorithm.

Definition

Multilevel Queue Scheduling is a CPU scheduling algorithm in which the ready queue is permanently divided into multiple queues based on process type, priority, or characteristics.

Each queue contains a specific class of processes and can use its own scheduling algorithm.

Key Property: Processes are PERMANENTLY assigned to a queue — no movement between queues.

Typical Queue Organization (Highest Priority → Lowest):

Queue	Process Type	Internal Algorithm	Priority
Queue 1 (Highest)	System Processes	Round Robin (Q=4ms)	Highest
Queue 2	Foreground / Interactive Processes	Round Robin (Q=8ms)	High
Queue 3 (Lowest)	Background / Batch Processes	FCFS	Low

Scheduling Among Queues — Two Methods

Method 1: Fixed Priority Preemptive Scheduling

Each queue is assigned a fixed priority ($Q1 > Q2 > Q3$).

CPU always executes the highest-priority non-empty queue.

Lower-priority queues run ONLY when all higher-priority queues are empty.

If a process arrives in a higher-priority queue, it IMMEDIATELY preempts the running lower-priority process.

Property	Detail
Advantage	Fast response for important/system processes
Disadvantage	Starvation — if system processes keep arriving, background processes may NEVER get CPU

Method 2: Time Slicing Among Queues

CPU time is divided among queues in fixed proportions.

Example: Q1 → 60% CPU time | Q2 → 30% CPU time | Q3 → 10% CPU time

Every queue receives guaranteed CPU time — starvation is avoided.

Property	Detail
Advantage	Fair CPU distribution — no starvation
Disadvantage	Slightly slower response for highest-priority queues compared to fixed-priority

Characteristics of MLQ

Characteristic	Detail
Queue Division	Ready queue permanently divided into multiple queues by process type
Internal Scheduling	Each queue uses its own scheduling algorithm independently
Process Assignment	Processes permanently assigned to one queue — no migration
Inter-queue Scheduling	Required: either fixed-priority or time-sliced among queues
Process Movement	■ No movement between queues — major limitation
Scheduling Levels	Two levels: within a queue AND among queues

Limitation of MLQ and Solution

Major Limitation: No Process Migration — a process placed in the Background Queue will ALWAYS remain there, even if its behavior changes later (e.g., a batch process becomes interactive).

Example: A process that was I/O bound becomes CPU bound — it deserves higher priority, but MLQ cannot accommodate this.

Solution: Multilevel Feedback Queue (MLFQ) Scheduling — processes CAN move between queues based on their observed behavior and history. This is the most general and practical CPU scheduling algorithm.

MLQ vs MLFQ — Quick Comparison

Property	MLQ	MLFQ
Process Migration	■ Not Allowed — permanent assignment	■ Allowed — moves based on behavior
Complexity	Simpler	More complex
Adaptability	Rigid — fixed categories	Adaptive — responds to runtime behavior
Starvation	Possible (fixed priority method)	Addressed via aging mechanisms
Practical Use	Less common	More common in real OS (Unix, Linux variants)
Queue Assignment	Based on process type at creation	Based on observed CPU usage patterns

QUICK REVISION — Key One-Liners & Interview Answers

OS Fundamentals

What is an OS?	Resource manager and control program between user and hardware.
What is a Kernel?	Core component of OS that manages hardware resources and provides services to user programs.
What is a Shell?	Interface between user and kernel (CLI or GUI).
What is a System Call?	Interface through which user programs request services from the kernel.
Why Dual Mode?	To provide protection — user programs cannot execute privileged operations.
Mode Bit: User Mode	Mode bit = 1
Mode Bit: Kernel Mode	Mode bit = 0
Most Important OS Service	Program Execution — without it, OS has no practical use.

Process Concepts

Process Definition	Program Under Execution = Program + Runtime Activity.
Process ≠ Program	5 Chrome windows = 1 program but 5 processes (instances).
Code Section	Read-only instructions. Fixed size after compilation.
Data Section	Global + Static variables (both have lifetime = entire process).
Heap	Dynamic memory. Grows at runtime. Size not fixed — depends on user input.
Stack	Local variables, function params, return addresses. Auto-freed on return.
PCB	Process identity card maintained by OS — contains all process attributes.
Context	Contents of PCB = Entire execution state of a process.
Scheduler	Selects next process. Dispatcher gives CPU and performs context switch.

Process States

New	Process being created.
Ready	Waiting for CPU — all resources allocated.
Running	Currently executing on CPU.
Waiting	Waiting for I/O or event — caused by process itself.

Terminated	Finished — resources released.
Preemption	OS forcibly takes CPU (time quantum expired or higher-priority arrived).
Waiting → Running?	NEVER directly. Must go Waiting → Ready → Running.
Suspended Ready	Ready but swapped to disk by Mid-Term Scheduler.
Suspended Blocked	Blocked and swapped to disk.

Schedulers

Long-Term Scheduler	Job Queue → Ready Queue. Controls MAX degree of multiprogramming. Runs rarely.
Short-Term Scheduler	Ready Queue → Running. Selects next process for CPU. Runs very frequently.
Mid-Term Scheduler	Performs swapping. REDUCES degree of multiprogramming.
Degree of Multiprogramming	Number of processes currently in RAM.

CPU Scheduling

FCFS	Non-preemptive. First arrived gets CPU. Problem: Convoy Effect.
SJF	Non-preemptive. Shortest BT first. Optimal avg WT. Problem: Starvation.
SRTF	Preemptive SJF. Smallest remaining time preempts. Better WT than SJF.
HRRN	Non-preemptive. $RR = (WT+BT)/BT$. Solves SJF starvation.
Priority	Higher priority first. Preemptive or non-preemptive. Starvation → Aging.
Round Robin	Preemptive. Time quantum Q. Fair. Very small Q → context switch overhead. Large Q → becomes FCFS.
Turnaround Time	$TAT = CT - AT = WT + BT$
Waiting Time	$WT = TAT - BT = CT - AT - BT$

Threads

Thread	Lightweight process — smallest unit of CPU execution inside a process.
Shared by threads	Code, Data, Heap, Files, OS Resources.
Unique per thread	Thread ID, Program Counter, Registers, Stack.
ULT vs KLT — Blocking	ULT: one blocks → whole process blocks. KLT: only that thread blocks.
Many-to-One	Multiple ULTs → one KLT. Fast but no parallelism.
One-to-One	One ULT ↔ one KLT. True parallelism. Used in Windows/Linux.

Many-to-Many	Multiple ULTs → fewer/equal KLTs. Best of both worlds.
--------------	--

Synchronization

Race Condition	Final result depends on execution order of concurrent processes.
Critical Section	Code segment accessing shared resources.
Mutual Exclusion	Only ONE process in CS at a time. Most important.
Progress	If CS empty and someone wants in, let them in immediately.
Bounded Waiting	No starvation — finite wait for every process.
Lock Variable	Fails Mutual Exclusion — check+set not atomic.
Turn Variable	Fails Progress — strict alternation blocks others.
Peterson's Solution	■ All three satisfied. Software solution for 2 processes only.
TestAndSet()	Hardware atomic. ■ ME ■ Progress ■ Bounded Waiting.
Swap()	Hardware atomic. ■ ME ■ Progress ■ Bounded Waiting.
Busy Waiting	Spinning in loop while waiting for lock — wastes CPU cycles.

MLQ Scheduling

MLQ Definition	Ready queue permanently divided into multiple queues by process type.
Process Migration in MLQ	■ Not allowed — processes permanently assigned.
Fixed Priority MLQ	Higher priority queue always preempts lower. Risk: starvation.
Time-Sliced MLQ	Each queue gets fixed % of CPU. No starvation.
MLQ Limitation	No adaptability — process stuck in wrong queue if behavior changes.
MLFQ Solution	Processes CAN move between queues based on runtime behavior.